2026-05-06-p1-f16-opentelemetry-integration-design

Phase 1 / Feature 16 — OpenTelemetry Integration

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

---

1. Goal

Ship real, end-to-end **OpenTelemetry tracing + metrics** for the HelixCode CLI agent. F16 wires an OTel `TracerProvider` and `MeterProvider` into the CLI bootstrap, decorates the three central hot paths (LLM provider calls, tool registry execution, agent loop iterations) with span + counter + histogram instrumentation, and routes telemetry to one of three exporters selected at startup via standard OTel environment variables: **OTLP/gRPC** (production), **OTLP/HTTP** (proxy-friendly), or **stdout** (development). When no exporter env var is set the telemetry stack collapses to a no-op TracerProvider + MeterProvider with zero observable cost — telemetry off by default.

Three concrete user surfaces ship together:

1. **Standard OTel env-var configuration** (Q4=B): `OTEL_EXPORTER_OTLP_ENDPOINT`, `OTEL_EXPORTER_OTLP_PROTOCOL` (`grpc` | `http/protobuf` | `stdout`), `OTEL_SERVICE_NAME` (default `"helixcode"`), `OTEL_RESOURCE_ATTRIBUTES` (comma-separated `k=v` , `k=v` per OTel spec). **No yaml/CLI flags in v1** — config is exclusively env-var driven so the OTel ecosystem's standard tooling (operators, sidecars, agents) configures HelixCode without HelixCode-specific knowledge.
2. **Three exporter implementations** (Q2=C):
   - **OTLP/gRPC** — selected when `OTEL_EXPORTER_OTLP_PROTOCOL=grpc` (or unset and endpoint looks gRPC-shaped). Production default.
   - **OTLP/HTTP** — selected when `OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf`. Proxy-friendly; used when corporate proxies block raw gRPC.
   - **stdout** — selected when `OTEL_EXPORTER_OTLP_PROTOCOL=stdout`. Writes spans + metrics as JSON-shaped lines to a configurable writer (default `os.Stderr`). Development-only.
3. `/telemetry` **slash command** (Q5=B): `status` (alias of bare `/telemetry` — exporter kind, endpoint, service name, span/metric counts since startup), `show` (last N exported spans/metrics from an in-memory ring buffer for live debugging — only populated when stdout exporter is active), `flush` (calls `ForceFlush(timeout=5s)` on TracerProvider + MeterProvider). **No cobra subcommand** — inspection and force-flush both go through the slash.

The instrumentation scope (Q3=B) is exactly three call sites:

- **LLM provider calls** — `Provider.Generate` and `Provider.GenerateStream`. One span per call (`llm.generate` / `llm.generate_stream`); a

`helixcode_llm_tokens_total` Int64Counter labelled by `model` + `direction` (`prompt`|`completion`); a `helixcode_llm_latency_seconds` Float64Histogram labelled by `model`. Decorated via a `TracedLLMProvider` wrapper at the **calling site** (registered into the agent and tool callers) — NOT by editing each of the four cloud providers (anthropic, bedrock, vertex, azure) plus ollama. Single integration point, zero per-provider churn.

- **Tool registry Execute** — `ToolRegistry.Execute(ctx, name, params)`. One span per call (`tool.execute`); a `helixcode_tool_calls_total` Int64Counter labelled by `tool` + `status` (`success`|`error`); a `helixcode_tool_latency_seconds` Float64Histogram labelled by `tool`. Instrumented at the single chokepoint in `internal/tools/registry.go::Execute` — same place F13 wires LSP auto-trigger and F08 wires plan-mode gating.
- **Agent loop iterations** — wherever the agent's outer loop iterates (the spec calls it "iteration" not "turn" because v1 BaseAgent runs a single LLM call per task; the "iteration" is the unit `executeTaskWithLLM` represents). One span per iteration (`agent.iteration`); a `helixcode_agent_iterations_total` Int64Counter; a `helixcode_agent_iteration_duration_seconds` Float64Histogram.

The scope of F16 is **TelemetryProvider + 3 exporters + 3 instrumentation sites + /telemetry slash + env-var config**. Logs-bridge from zap is **explicitly deferred to F16.5** (zap stays the logging layer; OTel stays the tracing+metrics layer; we do not bridge zap → OTel logs in v1 because the ecosystem's logs API is still in Beta as of writing). Sandbox/subagent/LSP/session/MCP instrumentation is also deferred to F16.5. Sampling configuration beyond the OTel env-var defaults is deferred. A Prometheus exporter is deferred (the porting doc references it; we cut it because OTLP + the OpenTelemetry Collector covers Prometheus end-to-end without a second exporter chain).

Anti-bluff: a "TelemetryProvider" that initialises an exporter but never produces a span, or that creates spans that the exporter never sees, or that wraps the LLM provider but never increments the counter, is a critical defect (§5.2). The single largest bluff vector for F16 is "telemetry on but pipeline broken" — looks correct from compilation alone, fails on any real export verification.

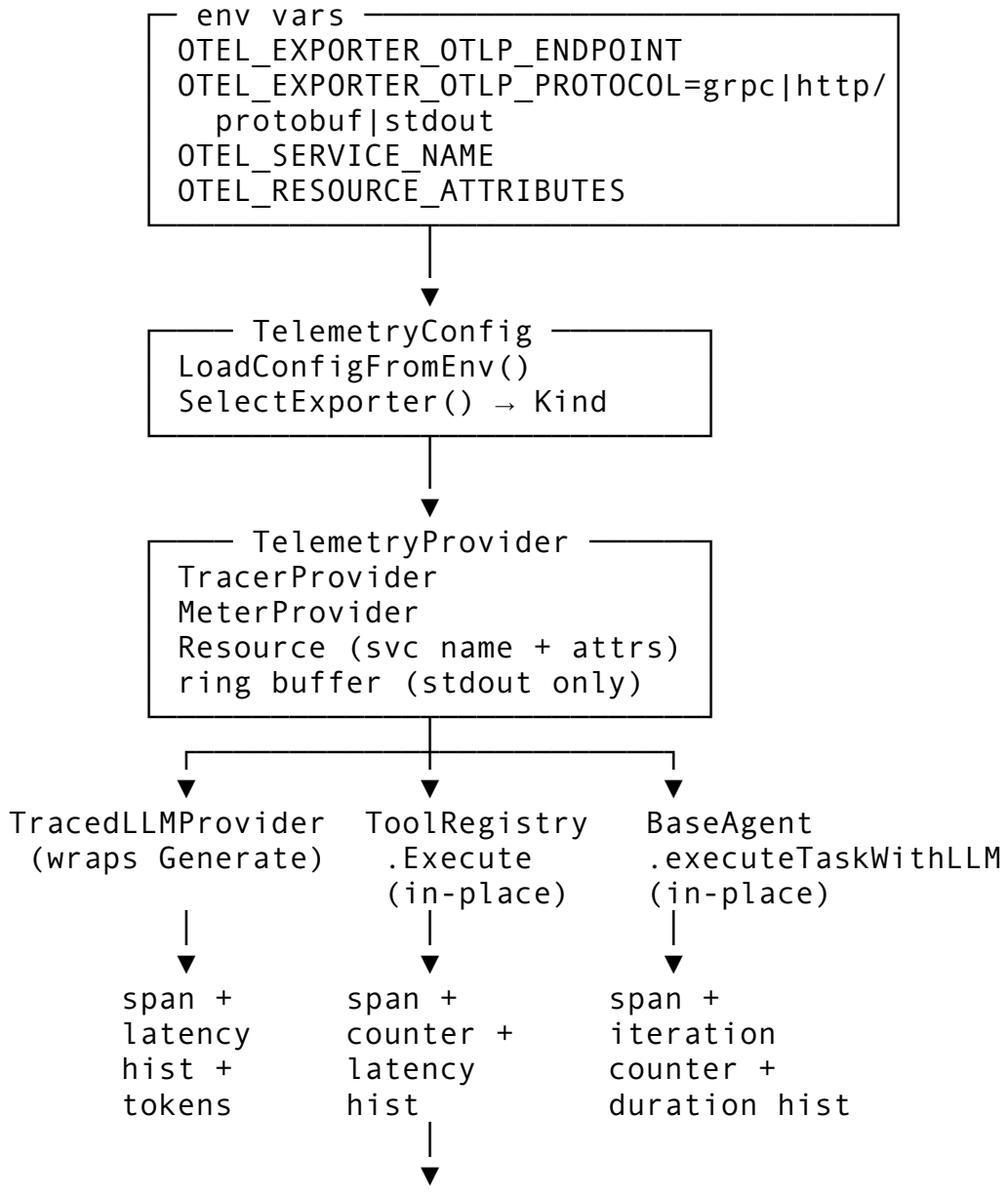
---

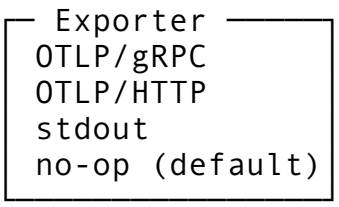
2. Architecture

Five layers, all under `internal/telemetry/`:

- `TelemetryConfig` — pure data: parsed env vars (`Endpoint`, `Protocol`, `ServiceName`, `ResourceAttributes`, `Insecure`, `Timeout`, `BlockedAttributeKeys`). Constructed by `LoadConfigFromEnv()`. When all OTEL_* env vars are unset, returns `TelemetryConfig{Enabled: false}` and the rest of the stack collapses to no-op providers.
- `ExporterKind` — string enum: `ExporterOTLPGRPC`, `ExporterOTLPHTTP`, `ExporterStdout`, `ExporterNone`. Decided by `TelemetryConfig.SelectExporter()` from `OTEL_EXPORTER_OTLP_PROTOCOL` + endpoint shape.
- `TelemetryProvider` — owns the pipeline. Holds a `*sdktrace.TracerProvider`, a `*sdkmetric.MeterProvider`, the chosen `ExporterKind`, a startup-time `*resource.Resource`, span/metric counters (for `/telemetry status`), and an in-memory ring buffer of last-N exported items (for `/telemetry show`, populated only by the stdout exporter wrapper — not by OTLP because stdout is already cheap and OTLP would double-buffer).

- **Instrumentation decorators** — three small types:
  - `TracedLLMProvider` wraps `llm.Provider` (struct embedding the inner Provider; overrides `Generate` + `GenerateStream` only; pass-through everything else).
  - **Tool-registry instrumentation** is **not** a wrapper — it's a 6-line in-place block at the start of `ToolRegistry.Execute` (mirrors F13's LSP auto-trigger placement). The registry holds an optional `*telemetry.TelemetryProvider`; when non-nil, Execute starts a span and records counter + histogram on return.
  - **Agent-loop instrumentation** is in-place inside `BaseAgent.executeTaskWithLLM` — wrap the existing `provider.Generate` call site with a span + iteration counter. Same `*telemetry.TelemetryProvider` injected via a new `BaseAgent.SetTelemetryProvider(*telemetry.TelemetryProvider)` setter (mirrors F01's `SetAutoCompactor`).
- `/telemetry` **slash command** — `internal/commands/telemetry_command.go` mirrors F14 `/sandbox` shape: defines a `TelemetryProviderInspector` interface in the commands package so the slash is testable with a fake while main.go passes the real `*telemetry.TelemetryProvider`.





Shutdown: at process exit, `TelemetryProvider.Shutdown(ctx)` calls `tp.ForceFlush(ctx)` then `tp.Shutdown(ctx)` then the same on the MeterProvider; deadline 5 s; errors are logged at zap-WARN, never fatal — telemetry must never crash the agent.

---

3. Components

3.1 New files

- `helix_code/internal/telemetry/types.go` — `TelemetryConfig`, `ExporterKind` enum, `TelemetryStats` (for /telemetry status), error sentinels. Exported `BlockedAttributeKeys` default list (CONST-042 secret-attribute blocklist; see §5.2).
- `helix_code/internal/telemetry/types_test.go`.
- `helix_code/internal/telemetry/config.go` — `LoadConfigFromEnv() *TelemetryConfig` + `SelectExporter() ExporterKind`. Pure; no OTel deps.
- `helix_code/internal/telemetry/config_test.go`.
- `helix_code/internal/telemetry/provider.go` — `TelemetryProvider` constructor (`NewTelemetryProvider(ctx, cfg, logger) (*TelemetryProvider, error)`), `Tracer()`, `Meter()`, `Shutdown(ctx)`, `ForceFlush(ctx)`, `Stats()`, `RingBuffer()` for /telemetry show.
- `helix_code/internal/telemetry/provider_test.go`.
- `helix_code/internal/telemetry/llm_instrumentation.go` — `TracedLLMProvider` decorator + factory.
- `helix_code/internal/telemetry/llm_instrumentation_test.go`.
- `helix_code/internal/telemetry/tool_instrumentation.go` — small `instrumentToolCall(ctx, tp, name) (context.Context, func(err error))` helper used in-place inside `ToolRegistry.Execute`. Lives in the telemetry package so the tool registry imports it cleanly.
- `helix_code/internal/telemetry/tool_instrumentation_test.go`.
- `helix_code/internal/telemetry/agent_instrumentation.go` — analogous helper for the agent loop iteration: `instrumentAgentIteration(ctx, tp) (context.Context, func(err error))`.
- `helix_code/internal/telemetry/agent_instrumentation_test.go`.
- `helix_code/internal/telemetry/attribute_filter.go` — `FilterAttributes([]attribute.KeyValue, blocked []string) []attribute.KeyValue`. CONST-042 secret-shaped-attribute filter (§5.2).
- `helix_code/internal/telemetry/attribute_filter_test.go`.
- `helix_code/internal/commands/telemetry_command.go` — `/telemetry` slash (`status`/`show`/`flush`).
- `helix_code/internal/commands/telemetry_command_test.go`.
- `helix_code/tests/integration/telemetry_test.go` — `//go:build integration` (gating per §5.2).
- `helix_code/tests/integration/cmd/p1f16_challenge/main.go` — runtime evidence harness with in-tree fake OTLP/HTTP receiver.

- `challenges/p1-f16-opentelemetry-integration/CHALLENGE.md` + `run.sh`.

3.2 Modified files

- `helix_code/go.mod` — add OTel deps (§3.5).
- `helix_code/internal/tools/registry.go` — add `telemetryProvider *telemetry.TelemetryProvider` field; add `SetTelemetryProvider(*telemetry.TelemetryProvider)` setter (mirrors `SetLSPManager` / `SetSandboxManager` / `SetSubagentManager`); wrap `Execute` body with `instrumentToolCall` (5 added lines).
- `helix_code/internal/agent/base_agent.go` — add `telemetryProvider *telemetry.TelemetryProvider` field; add `SetTelemetryProvider` setter; wrap `executeTaskWithLLM` LLM-call site with `instrumentAgentIteration` (5 added lines).
- `helix_code/cmd/cli/main.go` — three lines: load config from env, construct TelemetryProvider, register `/telemetry` slash + setters on registry/agent + `defer tp.Shutdown(ctx)`. The init goes alongside the existing logger construction (~line 400).

3.3 Types

```
// internal/telemetry/types.go

type ExporterKind string

const (
    ExporterNone     ExporterKind = "none"      // default: no
        env vars set → no-op providers
    ExporterStdout   ExporterKind = "stdout"
    ExporterOTLPGRPC ExporterKind = "otlp-grpc"
    ExporterOTLPHTTP ExporterKind = "otlp-http"
)

type TelemetryConfig struct {
    Enabled              bool              // false when no
        OTEL_* env var is set
    Kind                 ExporterKind
    Endpoint             string            //
        OTEL_EXPORTER_OTLP_ENDPOINT
    Protocol             string            //
        OTEL_EXPORTER_OTLP_PROTOCOL raw
    ServiceName          string            // OTEL_SERVICE_NAME
        (default "helixcode")
    ResourceAttributes   map[string]string //
        OTEL_RESOURCE_ATTRIBUTES parsed
    Insecure             bool              //
        OTEL_EXPORTER_OTLP_INSECURE=true
    Timeout              time.Duration     //
        OTEL_EXPORTER_OTLP_TIMEOUT (default 10s)
```

```
    BlockedAttributeKeys []string          // CONST-042
        blocklist; defaults applied when nil
}

func LoadConfigFromEnv() *TelemetryConfig
func (c *TelemetryConfig) SelectExporter() ExporterKind   //
        disambiguates from Protocol + Endpoint

// DefaultBlockedAttributeKeys is the seed list applied when
// TelemetryConfig.BlockedAttributeKeys is nil.  CONST-042-
        anchored.
var DefaultBlockedAttributeKeys = []string{
    "api_key", "apikey", "api-key",
    "token", "auth", "authorization", "bearer",
    "password", "passwd", "secret",
    "anthropic_api_key", "openai_api_key", "google_api_key",
    "aws_access_key_id", "aws_secret_access_key",
    "prompt",        // never emit full prompt text as a span
        attribute (CONST-042)
    "prompt_body",
    "request_body",
    "response_body",
}

type TelemetryStats struct {
    Kind          ExporterKind
    Endpoint      string
    ServiceName   string
    SpansStarted  uint64
    SpansEnded    uint64
    MetricsRecord uint64
    StartTime     time.Time
    LastFlushAt   time.Time
}

var (
    ErrTelemetryDisabled    =
        errors.New("telemetry: disabled (no OTEL_* env vars
        set)")
    ErrUnknownExporterKind  = errors.New("telemetry: unknown
        exporter kind")
    ErrExporterUnreachable  = errors.New("telemetry: exporter
        endpoint unreachable")
)

// internal/telemetry/provider.go

type TelemetryProvider struct {
    cfg          *TelemetryConfig
```

```
    tracerProv   *sdktrace.TracerProvider
    meterProv    *sdkmetric.MeterProvider
    tracer       trace.Tracer            // cached:
        tracerProv.Tracer("helixcode")
    meter        metric.Meter            // cached:
        meterProv.Meter("helixcode")
    resource     *resource.Resource
    logger       *zap.Logger

    // pre-built instruments (allocated once at construction)
    llmTokensCounter      metric.Int64Counter
    llmLatencyHist        metric.Float64Histogram
    toolCallsCounter      metric.Int64Counter
    toolLatencyHist       metric.Float64Histogram
    agentIterCounter      metric.Int64Counter
    agentIterDurationHist metric.Float64Histogram

    stats        TelemetryStats
    statsMu      sync.RWMutex
    ringBuf      *ringBuffer        // populated only when Kind
        == ExporterStdout
}

func NewTelemetryProvider(ctx context.Context, cfg
        *TelemetryConfig, logger *zap.Logger)
        (*TelemetryProvider, error)
// Returns a non-nil *TelemetryProvider with no-op providers when
        cfg.Enabled == false (zero-cost path).

func (t *TelemetryProvider) Tracer() trace.Tracer
func (t *TelemetryProvider) Meter() metric.Meter
func (t *TelemetryProvider) Stats() TelemetryStats
func (t *TelemetryProvider) RingBuffer()
        []ExportedRecord       // empty unless stdout exporter
func (t *TelemetryProvider) ForceFlush(ctx context.Context) error
func (t *TelemetryProvider) Shutdown(ctx context.Context)
        error  // ForceFlush + Shutdown both providers; deadline
        5s

// IsNoOp reports whether telemetry is disabled
        (TelemetryProvider was constructed
// with cfg.Enabled == false). Callers use this to skip work that
        only matters
// when a real exporter is wired.
func (t *TelemetryProvider) IsNoOp() bool

// internal/telemetry/llm_instrumentation.go

type TracedLLMProvider struct {
```

```
    inner llm.Provider           // pass-through everything
        except Generate/GenerateStream
    tp    *TelemetryProvider     // nil → pass-through (no
        instrumentation)
}

func NewTracedLLMProvider(inner llm.Provider, tp
        *TelemetryProvider) *TracedLLMProvider

// All non-Generate methods delegate to inner unchanged.
func (t *TracedLLMProvider) GetType() llm.ProviderType         {
        return t.inner.GetType() }
func (t *TracedLLMProvider) GetName() string                   {
        return t.inner.GetName() }
func (t *TracedLLMProvider) GetModels() []llm.ModelInfo        {
        return t.inner.GetModels() }
// ...
func (t *TracedLLMProvider) Generate(ctx context.Context, req
        *llm.LLMRequest) (*llm.LLMResponse, error)
func (t *TracedLLMProvider) GenerateStream(ctx context.Context,
        req *llm.LLMRequest, ch chan<- llm.LLMResponse) error

// internal/telemetry/tool_instrumentation.go

// instrumentToolCall starts a `tool.execute` span + records a
        counter increment and a
// latency histogram on the returned closer. Used in-place by
        ToolRegistry.Execute:
//
//     ctx, end := telemetry.InstrumentToolCall(ctx,
        r.telemetryProvider, name)
//     defer func() { end(execErr) }()
func InstrumentToolCall(ctx context.Context, tp
        *TelemetryProvider, toolName string) (context.Context,
        func(error))
func InstrumentAgentIteration(ctx context.Context, tp
        *TelemetryProvider) (context.Context, func(error))

// internal/commands/telemetry_command.go

type TelemetryProviderInspector interface {
    Stats() telemetry.TelemetryStats
    RingBuffer() []telemetry.ExportedRecord
    ForceFlush(ctx context.Context) error
    IsNoOp() bool
}

type TelemetryCommand struct { tp TelemetryProviderInspector }
```

```
func NewTelemetryCommand(tp TelemetryProviderInspector)
        *TelemetryCommand
func (c *TelemetryCommand) Name() string         { return
        "telemetry" }
func (c *TelemetryCommand) Aliases() []string    { return nil }
func (c *TelemetryCommand) Description() string  { return
        "Inspect telemetry exporter status, last exported items,
        or force-flush the pipeline." }
func (c *TelemetryCommand) Usage() string        { return "/
        telemetry [status|show|flush]" }
func (c *TelemetryCommand) Execute(ctx context.Context, cc
        *CommandContext) (*CommandResult, error)
```

3.4 User surfaces

Environment-variable configuration:

Env var	Required?	Notes
`OTEL_EXPORTER_OTLP_ENDPOINT`	required to enable telemetry	URL form: `host:4317` for gRPC, `http://host:4318` for HTTP, ignored for `stdout`
`OTEL_EXPORTER_OTLP_PROTOCOL`	optional	`grpc` (default), `http/protobuf`, `stdout`
`OTEL_SERVICE_NAME`	optional	default `"helixcode"`
`OTEL_RESOURCE_ATTRIBUTES`	optional	comma-separated `k=v`, `k=v` per OTel spec
`OTEL_EXPORTER_OTLP_INSECURE`	optional	`true` to skip TLS (dev only)
`OTEL_EXPORTER_OTLP_TIMEOUT`	optional	seconds, default 10

Exporter selection rule: 1. If `OTEL_EXPORTER_OTLP_PROTOCOL=stdout` → `ExporterStdout`. 2. Else if `OTEL_EXPORTER_OTLP_ENDPOINT` is unset and `OTEL_EXPORTER_OTLP_PROTOCOL` is unset → `ExporterNone` (no-op providers, zero-cost path). 3. Else if `OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf` → `ExporterOTLPHTTP`. 4. Else (default) → `ExporterOTLPGRPC`.

Slash command `/telemetry`: - `/telemetry` (alias of `status`) — table: `KIND ENDPOINT SERVICE SPANS METRICS UPTIME`. When `IsNoOp()`, prints `telemetry disabled (set OTEL_EXPORTER_OTLP_ENDPOINT or OTEL_EXPORTER_OTLP_PROTOCOL=stdout).` - `/telemetry show` — last 10 exported items from the ring buffer (only populated when the stdout exporter is active; OTLP exporters do NOT mirror into the ring buffer because they round-trip to a real backend already). - `/telemetry flush` — calls `ForceFlush(ctx)` with a 5 s deadline; prints `flushed N spans, M metrics in <dur>`. On error, prints `flush failed: <err>` and the slash returns success=false.

No cobra subcommand (Q5=B). Inspection and force-flush both go through the slash command.

3.5 New external dependencies (exact versions)

OTel Go SDK is on a synchronised release train. Pin all entries to a single major version (`v1.30.x` for trace/SDK + `v1.30.x`-aligned metric/exporter modules; the metric SDK + exporters in the v1.30 train carry the matching `v1.30.0` tag — no separate `v0.x` line for metric exporters as of the v1.30 release):

```
go.opentelemetry.io/otel                                          v1.3
go.opentelemetry.io/otel/sdk                                      v1.3
go.opentelemetry.io/otel/sdk/metric                               v1.3
go.opentelemetry.io/otel/metric                                   v1.3
go.opentelemetry.io/otel/trace                                    v1.3
go.opentelemetry.io/otel/exporters/otlp/otlptrace                 v1.3
go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc   v1.3
go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracehttp   v1.3
go.opentelemetry.io/otel/exporters/otlp/otlpmetric                v1.3
go.opentelemetry.io/otel/exporters/otlp/otlpmetric/otlpmetricgrpc  v1.3
go.opentelemetry.io/otel/exporters/otlp/otlpmetric/otlpmetrichttp  v1.3
go.opentelemetry.io/otel/exporters/stdout/stdouttrace             v1.3
go.opentelemetry.io/otel/exporters/stdout/stdoutmetric            v1.3
```

Plus transitive: `google.golang.org/grpc`, `google.golang.org/protobuf`, `go.opentelemetry.io/proto/otlp`, `github.com/cenkalti/backoff/v4`, `github.com/grpc-ecosystem/grpc-gateway/v2` (already partially present for gRPC code paths). Estimated transitive footprint: ~25 new modules in `go.sum`. No CGO.

T02 runs `go mod tidy` and freezes `go.sum`; the resulting diff is reviewed for spurious major-version drifts before commit.

3.6 Existing-code constraints

- `internal/llm/missing_types.go::Provider` interface has 11 methods. `TracedLLMProvider` embeds the inner Provider via Go struct embedding so only the 2 methods F16 instruments (`Generate` + `GenerateStream`) need explicit definitions; the remaining 9 (`GetType`, `GetName`, `GetModels`, `GetCapabilities`, `IsAvailable`, `GetHealth`, `Close`, `GetContextWindow`, `CountTokens`) are promoted automatically. This keeps the wrapper compact and prevents drift if the Provider interface grows.
- `internal/tools/registry.go::Execute` already has F07 (background dispatch), F08 (plan-mode gate), F13 (LSP auto-trigger) hooks. F16 instrumentation slots in immediately after `tool, err := r.Get(name)` and wraps the entire `tool.Execute(ctx, params)` call site so the span captures hook overhead too. Ordering: F07 dispatch first (it short-circuits to a different code path), then plan-mode gate (it can short-circuit with an error), then **telemetry span open** (so the span captures the actual execution + LSP trigger), then tool.Execute, then hooks-after, then **telemetry span close**, then LSP trigger. This means the span timing represents wall-time the user perceives.
- `internal/agent/base_agent.go::executeTaskWithLLM` is the v1 agent-iteration site. F16 wraps the existing `provider.Generate` call (line 389) inside `instrumentAgentIteration`. The agent loop in v1 is a single LLM call per task; multi-iteration loops are part of the broader planner work and v1 instrumentation captures one iteration per task — documented in the Stats output as `iterations` not `turns`.
- `cmd/cli/main.go` already wires `subagent.IsSubagentInvocation` (F15) and `sandbox.IsHelperInvocation` (F14) at the top of `main()`. Telemetry init does NOT need helper-mode-style early dispatch because subagent helpers and sandbox helpers

do NOT inherit telemetry — they run a single short-lived task and exit; instrumenting them is F16.5 work. The parent-process telemetry init slots in alongside the logger construction (~line 400), well after the helper checks.

4. Data flow

4.1 Startup wiring (`cmd/cli/main.go`)

```
main()
  ├─ subagent.IsSubagentInvocation() check (F15, FIRST)
  ├─ sandbox.IsHelperInvocation() check (F14, SECOND)
  ├─ … existing CLI bootstrap (cobra, config load, logger) …
  │
  ├─ telCfg := telemetry.LoadConfigFromEnv()
  ├─ tp, err := telemetry.NewTelemetryProvider(ctx, telCfg, logger)
  ├─ if err != nil:
  │     logger.Warn("telemetry init failed; continuing without telemetry",
  │     tp = nil    // registry/agent setters tolerate nil
  ├─ defer func() {
  │     if tp != nil {
  │         shCtx, sc := context.WithTimeout(context.Background(), 5*time.
  │         defer sc()
  │         _ = tp.Shutdown(shCtx)
  │     }
  │   }()
  │
  ├─ … existing provider := factory(...) …
  ├─ if tp != nil && !tp.IsNoOp():
  │     provider = telemetry.NewTracedLLMProvider(provider, tp)
  │
  ├─ … existing toolReg := tools.NewToolRegistry(...) …
  ├─ if tp != nil: toolReg.SetTelemetryProvider(tp)
  ├─ if tp != nil: baseAgent.SetTelemetryProvider(tp)
  │
  ├─ slashRegistry.Register(commands.NewTelemetryCommand(tp))   // tp may
  └─ … rest of bootstrap …
```

Failure modes that MUST NOT crash the agent: - `OTEL_EXPORTER_OTLP_ENDPOINT` is set but unreachable → `NewTelemetryProvider` succeeds (BatchSpanProcessor + OTLP exporter are async; transient connection failures get retried internally and surface only on `ForceFlush`/`Shutdown`). The agent runs normally; spans accumulate locally; on shutdown the deadline-bounded flush either ships them or logs a warning. - `OTEL_EXPORTER_OTLP_PROTOCOL=bogus` → `LoadConfigFromEnv` returns `ErrUnknownExporterKind`; main.go logs WARN; `tp = nil`; agent runs without telemetry. - `NewTelemetryProvider` panics in the OTel SDK (defensive) → `recover`-wrapped at the boundary; logged at WARN; `tp = nil`.

4.2 LLM call instrumentation flow

```
TracedLLMProvider.Generate(ctx, req)
  ├─ if t.tp == nil || t.tp.IsNoOp(): return t.inner.Generate(ctx, req)
  ├─ start := time.Now()
  ├─ ctx, span := t.tp.Tracer().Start(ctx, "llm.generate",
```

```
   │     trace.WithAttributes(
   │         attribute.String("llm.model",       req.Model),
   │         attribute.Int   ("llm.max_tokens",  req.MaxTokens),
   │         attribute.Int   ("llm.message_count", len(req.Messages)),
   │         // NB: DO NOT add attribute.String("llm.prompt", …) — CONST-04
   │     ))
   ├─ resp, err := t.inner.Generate(ctx, req)
   ├─ dur := time.Since(start).Seconds()
   ├─ t.tp.observeLLMLatency(req.Model, dur)
   ├─ if resp != nil:
   │     t.tp.addLLMTokens(req.Model, "prompt",     int64(resp.Usage.Prompt
   │     t.tp.addLLMTokens(req.Model, "completion", int64(resp.Usage.Comple
   │     span.SetAttributes(
   │         attribute.Int("llm.usage.prompt_tokens",     resp.Usage.Prompt
   │         attribute.Int("llm.usage.completion_tokens", resp.Usage.Comple
   │         attribute.Int("llm.usage.total_tokens",      resp.Usage.TotalT
   │         attribute.String("llm.finish_reason",        resp.FinishReason
   │     )
   ├─ if err != nil:
   │     span.RecordError(err)
   │     span.SetStatus(codes.Error, err.Error())
   ├─ span.End()
   └─ return resp, err
```

`GenerateStream` follows the same shape with the span ending after `ch` is closed.

4.3 Tool call instrumentation flow (in-place inside `ToolRegistry.Execute`)

```
ToolRegistry.Execute(ctx, name, params)
   ├─ if run_in_background: dispatch & return (existing F07 path; NOT instr
   ├─ checkPlanModeGate (existing F08; NOT inside the span — its rejection
   ├─ tool, err := r.Get(name); if err: return err   (NOT inside the span —
   ├─ tool.Validate(...); if err: return err          (NOT inside the span)
   ├─ // F16: open span + start clock
   ├─ ctx, end := telemetry.InstrumentToolCall(ctx, r.telemetryProvider, na
   ├─ var execErr error
   ├─ defer func() { end(execErr) }()
   ├─ … existing fireBefore + tool.Execute + fireAfter + plan-action-mark +
   └─ return result, execErr
```

`InstrumentToolCall(ctx, tp, name)`: - Returns `ctx` unchanged + a no-op closer when `tp == nil || tp.IsNoOp()`. - Else: starts span `tool.execute` with attribute `tool.name=<name>`. The closer increments `helixcode_tool_calls_total{tool=<name>, status=success|error}`, records `helixcode_tool_latency_seconds{tool=<name>}`, calls `span.RecordError` + `span.SetStatus(codes.Error, …)` on error, ends the span.

4.4 Agent iteration flow (in-place inside `BaseAgent.executeTaskWithLLM`)

```
executeTaskWithLLM(ctx, t)
   ├─ … existing prompt build, model select, request construct, auto-compac
```

```
   ├─ // F16: open iteration span + start clock
   ├─ ctx, end := telemetry.InstrumentAgentIteration(ctx, a.telemetryProvid
   ├─ var llmErr error
   ├─ defer func() { end(llmErr) }()
   ├─ response, llmErr := a.llmProvider.Generate(ctx, request)
   ├─ if llmErr != nil: dispatchOnError; return nil, llmErr
   ├─ result, err := a.processLLMResponse(ctx, t, response)
   └─ return result, err
```

`InstrumentAgentIteration` mirrors `InstrumentToolCall`: span `agent.iteration` + counter + duration histogram. The span ends after `processLLMResponse` returns — so an iteration's wall-time covers the LLM call AND the response processing (parse, code-application). This is intentional: it gives users the full per-iteration latency they care about.

4.5 Shutdown flow

```
defer (in main.go)
   ├─ if tp == nil: return
   ├─ ctx, cancel := context.WithTimeout(context.Background(), 5*time.Secon
   ├─ defer cancel()
   ├─ if err := tp.Shutdown(ctx); err != nil:
   │     logger.Warn("telemetry shutdown failed", zap.Error(err))
   └─ // process exits

TelemetryProvider.Shutdown(ctx)
   ├─ tp.statsMu.Lock; tp.stats.LastFlushAt = time.Now(); tp.statsMu.Unlock
   ├─ var errs []error
   ├─ if t.tracerProv != nil:
   │     errs = append(errs, t.tracerProv.ForceFlush(ctx))
   │     errs = append(errs, t.tracerProv.Shutdown(ctx))
   ├─ if t.meterProv != nil:
   │     errs = append(errs, t.meterProv.ForceFlush(ctx))
   │     errs = append(errs, t.meterProv.Shutdown(ctx))
   └─ return errors.Join(errs...)
```

`ForceFlush` is called from `/telemetry flush` with the same 5 s deadline.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **No env vars set** — `LoadConfigFromEnv` returns `&TelemetryConfig{Enabled: false, Kind: ExporterNone}`. `NewTelemetryProvider` returns a non-nil `*TelemetryProvider` whose Tracer is `noop.NewTracerProvider().Tracer("helixcode")` and whose Meter is `metricnoop.NewMeterProvider().Meter("helixcode")`. All instrumentation helpers detect `IsNoOp()` and short-circuit to zero-allocation pass-through. `/telemetry status` reports `disabled`.
- **Unknown protocol** — `LoadConfigFromEnv` errors with `ErrUnknownExporterKind`; main.go logs WARN; runs with `tp = nil`.
- **Exporter unreachable at startup** — OTLP exporters construct asynchronously; `NewTelemetryProvider` succeeds. The first export attempt may fail; OTel SDK's

`BatchSpanProcessor` retries internally (configurable via `OTEL_EXPORTER_OTLP_TIMEOUT`). On `ForceFlush`/`Shutdown`, deadline-bounded errors are logged at WARN and the process exits cleanly.
- **Exporter unreachable mid-run** — same: the BSP buffers; if the buffer fills (default 2048 spans), older spans are dropped silently. v1 does NOT add backpressure to the LLM call; the agent must remain usable when the collector is down.
- `Shutdown` **deadline exceeded** — `tp.Shutdown` returns the wrapped deadline error; main.go logs WARN; process exits 0 (telemetry must never gate exit code).
- `ForceFlush` **from** `/telemetry flush` **deadline exceeded** — slash command returns success=false with `flush timed out after 5s — collector may be unreachable`.
- **OTel SDK panic** — every public method on `TelemetryProvider` defers a `recover()` that downgrades panic to a logged WARN. Telemetry must never crash the host process.
- **TracedLLMProvider with nil inner** — constructor returns `nil, errors.New("inner provider must not be nil")`; main.go aborts wrapping but continues running with the unwrapped provider.

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single largest bluff vector for F16 is "telemetry on but pipeline broken" — the SDK initialises, the exporter constructs, instrumentation appears to fire, but no spans actually reach a backend. This compiles, passes naive unit tests, and silently produces zero observability. Common bluff variants:

1. **(a) Tracer initialised but no spans actually created** — code path `tp.Tracer().Start(...)` is never reached (e.g., a guard `if tp != nil && tp.IsNoOp()` accidentally inverts the IsNoOp check; or the wrapper isn't wired into main.go).
2. **(b) Spans created but never exported** — TracerProvider's BatchSpanProcessor was constructed but `ForceFlush` is never called (e.g., the deferred shutdown is in the wrong scope; or shutdown uses a 0-deadline ctx).
3. **(c) Metrics registered but values never recorded** — the Counter is created in `NewTelemetryProvider` but the Add() call is in dead code (e.g., behind a feature flag that's always false).
4. **(d) Counters incremented but exporter never sees them** — MeterProvider's PeriodicReader was wired to the wrong exporter, or the exporter's `Export` method is never invoked because no MeterProvider was actually `SetMeterProvider`'d into otel global (the global is irrelevant — we use the local Meter — but a careless integration test that asserts via the global will silently green).

Required real-execution criteria (these define what "telemetry works" means in F16):

1. **Unit tests** — mocks OK at the exporter boundary. Tests verify that **spans are actually created with the right attributes** (use OTel's `tracetest.SpanRecorder`) and **metrics are actually recorded with the right labels** (use OTel's `metric/sdk/metricdata` reader pattern). Anti-bluff (a) + (c).
2. **Integration tests** (`-tags=integration`) — three scenarios, gating documented loudly:
    - **Stdout exporter test** — ALWAYS runs. Sets `OTEL_EXPORTER_OTLP_PROTOCOL=stdout`, runs an in-process LLM call via `subagent.FakeLLMProvider` (the F15 test provider — already in the integration-test build linkage), captures the stdout exporter's writer, asserts the

captured bytes contain the literal `"Name":"llm.generate"` AND a `helixcode_llm_tokens_total` metric record. Anti-bluff (b) + (d).
    - **OTLP/gRPC test** — gated on `OTEL_EXPORTER_OTLP_ENDPOINT` being set to a real reachable collector AND `OTEL_EXPORTER_OTLP_PROTOCOL=grpc`. SKIP-OK with marker `SKIP-OK: P1-F16 OTEL_EXPORTER_OTLP_ENDPOINT unset (export OTEL_EXPORTER_OTLP_ENDPOINT=localhost:4317 OTEL_EXPORTER_OTLP_PROTOCOL=grpc)` otherwise.
    - **OTLP/HTTP test** — gated on the same env vars with `OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf`. SKIP-OK marker analogous.
3. **Challenge harness** — exercises the stdout exporter end-to-end (always-runs phase) PLUS an **in-tree fake OTLP/HTTP receiver** (a tiny `net/http.Server` that registers `POST /v1/traces` + `POST /v1/metrics`, decodes the protobuf body, records the count of received spans + metrics, and exits with non-zero if the count is 0). The harness sets `OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:<port>` + `OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf`, runs a full agent call, force-flushes, asserts the receiver got ≥ 1 trace POST AND ≥ 1 metric POST. Real export, real HTTP, real protobuf.
4. **Challenge MUST exit non-zero if no spans were observed** in any phase that did run. Span-count-zero is treated as a hard failure, not a warning.

Concrete forbidden phrases (anti-bluff smoke):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/telemetry internal/commands/telemetry_command.go \
  && echo BLUFF || echo clean
```

Must always print `clean`.

CONST-042 secret-attribute filter (mandatory):

Span attributes and metric labels MUST NOT carry secret-shaped values. The mechanism: - `attribute_filter.go` exports `FilterAttributes(attrs []attribute.KeyValue, blocked []string) []attribute.KeyValue`. The function drops any KV whose **key** matches any string in `blocked` (case-insensitive, exact match). - The default blocklist `DefaultBlockedAttributeKeys` covers credential-shaped keys (`api_key`, `token`, `bearer`, `password`, `secret`, `authorization`, provider-specific `anthropic_api_key`, `openai_api_key`, AWS credential pair) plus prompt-body keys (`prompt`, `prompt_body`, `request_body`, `response_body`). - Every instrumentation helper that takes user-provided attributes (currently: tool-params attributes; not used in v1 but the seam is reserved for F16.5) MUST funnel through `FilterAttributes` before calling `span.SetAttributes`. v1's three instrumentation sites use only well-known model-metadata + token counts + finish reason, so the filter is preventative; the test still asserts the filter would drop a deliberately-injected `api_key=sk-abc` attribute. - Prompt body is NEVER added as a span attribute under any circumstance — the LLM-instrumentation flow above explicitly omits it. A unit test asserts: `TestTracedLLMProvider_DoesNotEmitPromptBodyAttribute`. - `BlockedAttributeKeys` is overridable via `TelemetryConfig.BlockedAttributeKeys` — additional org-specific keys can be appended at startup if a downstream user vendors HelixCode (not exposed via env var in v1; documented seam for F16.5).

Subprocess subagent inheritance: F15's subprocess subagent helper-mode runs `RunAsSubagent` which constructs its own runtime independently of the parent. v1 does NOT

pass telemetry through to subagent helpers (deferred to F16.5). The subagent's LLM call is therefore not traced; this is documented loudly in §8 and the porting doc cross-reference.

Real-execution criteria summary (each tied to a unit + Challenge phase):

Criterion	Unit	Integration
(1) spans created	`TestTracedLLMProvider_StartsSpan`	stdout: bytes cor `"Name":"llm`
(2) spans exported	`TestProvider_ForceFlushDrainsBSP`	stdout: bytes are
(3) metrics recorded	`TestTracedLLMProvider_AddsTokensCounter`	stdout: bytes cor `helixcode_l`
(4) metrics exported	`TestProvider_MetricsForceFlushDrainsReader`	OTLP gated: col metric record
(5) no secret-shaped attrs	`TestFilterAttributes_DropsBlockedKeys` + `TestTracedLLMProvider_DoesNotEmitPromptBodyAttribute`	n/a

The Challenge harness's fake-receiver exit-code logic uses positive evidence: `if observed_traces == 0 || observed_metrics == 0: exit 1`. Absence-of-error is NEVER acceptable — a Challenge that reports PASS without observed positive evidence is itself a bluff.

6. Testing

6.1 Unit (mocks OK at the exporter boundary)

- `TestExporterKind_String` — enum sanity.

- TestLoadConfigFromEnv_NoEnv_DisablesTelemetry.
- TestLoadConfigFromEnv_EndpointOnly_DefaultsToOTLPGRPC.
- TestLoadConfigFromEnv_ProtocolStdout_SelectsStdout.
- TestLoadConfigFromEnv_ProtocolHTTP_SelectsOTLPHTTP.
- TestLoadConfigFromEnv_ServiceNameDefault_helixcode.
- TestLoadConfigFromEnv_ResourceAttributes_ParsedAsCSV.
- TestLoadConfigFromEnv_BadProtocol_ReturnsErrUnknownExporterKind.
- TestSelectExporter_StdoutBeatsEndpointInference.
- TestNewTelemetryProvider_NoEnv_ReturnsNoOpProvider — IsNoOp() true; Tracer + Meter are functional but emit nothing.
- TestNewTelemetryProvider_StdoutKind_ReturnsRealProvider — IsNoOp() false.
- TestProvider_ForceFlushDrainsBSP — uses tracetest.SpanRecorder as the exporter; asserts spans flushed.
- TestProvider_MetricsForceFlushDrainsReader — uses metric/sdk/metricdata to read out the registered counter.
- TestProvider_Shutdown_RespectsCtxDeadline — passes a context.WithTimeout(0); asserts return within 100ms.
- TestFilterAttributes_DropsBlockedKeys — asserts api_key/token/secret/prompt entries are dropped.
- TestFilterAttributes_CaseInsensitive — API_KEY, Api-Key, apikey all dropped.
- TestFilterAttributes_PassesThroughSafeKeys — model, tool.name, llm.usage.prompt_tokens survive.
- TestTracedLLMProvider_StartsSpan — tracetest.SpanRecorder sees a span named llm.generate after a Generate call.
- TestTracedLLMProvider_AddsTokensCounter — manual metric.Reader exposes the counter increment.
- TestTracedLLMProvider_PassesThroughOtherMethods — GetType, GetModels, etc. delegate correctly via embedding.
- TestTracedLLMProvider_DoesNotEmitPromptBodyAttribute — assert the captured span has no attribute key in {"prompt", "prompt_body", "messages"}.
- TestTracedLLMProvider_RecordsErrorOnFailure — inner provider returns err; span's status code is codes.Error.
- TestTracedLLMProvider_NoOpFastPath — when tp.IsNoOp(), no allocations from the wrapper (assert via testing.AllocsPerRun < 1).
- TestInstrumentToolCall_NilProvider_NoOp — tp == nil → returns same ctx and a no-op closer.
- TestInstrumentToolCall_ErrorPath_RecordsErrorAndStatus.
- TestInstrumentToolCall_SuccessPath_RecordsSuccessLabel.
- TestInstrumentAgentIteration_NilProvider_NoOp.
- TestInstrumentAgentIteration_RecordsDuration.
- TestTelemetryCommand_Status_RendersTable.
- TestTelemetryCommand_Status_NoOpReportsDisabled.
- TestTelemetryCommand_Show_NoOpReportsRingBufferOnlyForStdout.
- TestTelemetryCommand_Flush_DelegatesToProviderForceFlush.
- TestTelemetryCommand_Flush_DeadlineErrorReportedToUser.
- TestTelemetryCommand_NilProvider_ReportsUnavailable — symmetric with /sandbox and /subagents patterns.

6.2 Integration (`//go:build integration`)

- `TestTelemetry_StdoutExporter_RealLLMCall_FakeProvider` — ALWAYS runs. Sets stdout protocol; runs a `subagent.FakeLLMProvider` Generate via TracedLLMProvider; force-flushes; asserts captured bytes contain `"Name":"llm.generate"` AND `helixcode_llm_tokens_total`.
- `TestTelemetry_OTLPGRPC_RealCollector` — gated on `OTEL_EXPORTER_OTLP_ENDPOINT` + `PROTOCOL=grpc`. SKIP-OK marker: `SKIP-OK: P1-F16 OTEL_EXPORTER_OTLP_ENDPOINT unset (export OTEL_EXPORTER_OTLP_ENDPOINT=localhost:4317 OTEL_EXPORTER_OTLP_PROTOCOL=grpc)`. Connects, exports, force-flushes, no errors.
- `TestTelemetry_OTLPHTTP_RealCollector` — analogous gating + assertion path.
- `TestTelemetry_ToolRegistry_InstrumentedExecute` — runs the registry against a real bash tool with the stdout exporter, asserts captured span name `tool.execute` and label `tool="bash"`.
- `TestTelemetry_AgentIteration_RealLLMCall_StdoutExport` — runs `BaseAgent.executeTaskWithLLM` with `FakeLLMProvider` + stdout exporter; asserts span name `agent.iteration` appears and counter increments.
- `TestTelemetry_NoOp_FastPath_RealRegistry` — telemetry disabled; runs 1000 tool calls; asserts execution time stays within 5% of the un-instrumented baseline (a regression-detector for accidental work in the no-op path).

6.3 Challenge (`challenges/p1-f16-opentelemetry-integration/`)

Five-phase output skeleton:

```
=== STDOUT EXPORTER (always runs) ===
[PASS] stdout: TelemetryProvider initialised with kind=stdout
[PASS] stdout: TracedLLMProvider.Generate produced a span (name=llm.genera
[PASS] stdout: helixcode_llm_tokens_total metric appeared in stdout bytes
[PASS] stdout: ToolRegistry.Execute produced span name=tool.execute
[PASS] stdout: agent.iteration span observed
[PASS] stdout: ForceFlush returned within 5s

=== FAKE OTLP/HTTP RECEIVER (always runs) ===
[PASS] otlp-http: in-tree receiver listening on http://127.0.0.1:<port>
[PASS] otlp-http: agent round-trip completed without error
[PASS] otlp-http: receiver observed >=1 POST /v1/traces (count=<n>)
[PASS] otlp-http: receiver observed >=1 POST /v1/metrics (count=<n>)
[PASS] otlp-http: decoded protobuf trace body had span name=llm.generate (

=== SECRET-ATTRIBUTE FILTER (always runs) ===
[PASS] filter: deliberately-injected attribute api_key=sk-abc-123 was drop
[PASS] filter: deliberately-injected attribute prompt=<long body> was drop
[PASS] filter: model=gpt-4 + tool.name=bash survived

=== NO-OP FAST PATH (always runs) ===
[PASS] noop: TelemetryProvider with no env vars reports IsNoOp=true
[PASS] noop: 1000 tool calls completed within latency budget vs baseline

=== REAL OTLP COLLECTOR (gated) ===
```

```
[PASS|skipped: OTEL_EXPORTER_OTLP_ENDPOINT unset (export OTEL_EXPORTER_OTL
[PASS] real-collector: TracerProvider.Shutdown returned without error
[PASS] real-collector: collector responded with success on at least 1 trac

SUMMARY: STDOUT=6/6 PASS; OTLP-HTTP=5/5 PASS; FILTER=3/3 PASS; NOOP=2/2 PA
```

The Challenge MUST exit non-zero on any assertion failure within phases that did run. Anti-bluff smoke clean check appended to harness output.

7. Cross-platform

OTel Go SDK is pure Go (no CGO) and supports `linux/amd64`, `linux/arm64`, `darwin/amd64`, `darwin/arm64`, `windows/amd64`. The OTLP/gRPC exporter pulls in `google.golang.org/grpc` which has a non-trivial dep footprint but compiles on all targets. `cross-compile linux/macos/windows` (the existing `make prod` target) is exercised to confirm. No `runtime.GOOS` switching needed in F16 code.

8. Out of scope (deferred)

- **Logs-bridge from zap to OTel logs** — F16.5. The OTel Go logs API entered Beta in 2024; we wait for stable release to avoid a churn cost. zap remains the logging surface in v1.
- **Subagent telemetry inheritance** (F15 subprocess subagent's LLM calls instrumented under the parent's trace ID) — F16.5. v1 subagents are not instrumented; their LLM calls produce no spans. Documented loudly in §5.2.
- **Sandbox / LSP / MCP / session / discovery instrumentation** — F16.5. v1 covers the three highest-signal hot paths only.
- **Custom OTel processors** (filtering/transformation pipelines beyond the simple BatchSpanProcessor + PeriodicReader) — F16.5.
- **Sampling configuration beyond OTel env-var defaults** — F16.5; v1 uses the SDK default `parent-based(traces-and-error-codes)` sampler. Users who need head sampling set `OTEL_TRACES_SAMPLER` per the OTel spec — supported transitively by the SDK without F16-specific code.
- **Prometheus exporter** — explicitly rejected in v1 (porting doc references it). OTLP + the OpenTelemetry Collector covers the Prometheus-shaped consumer end-to-end without HelixCode owning a second exporter chain.
- **Jaeger native exporter** — rejected (Jaeger now ingests OTLP natively; the Jaeger-format exporter is deprecated upstream).
- **W3C Trace Context propagation in/out of HTTP requests HelixCode makes to LLM providers** — out of v1 because the existing provider clients don't take a context-carrier seam. F16.5.
- `/telemetry config` **subcommand to inspect parsed TelemetryConfig** — covered today by `OTEL_*` env-var standard tooling; v1 surface area kept minimal.
- `TelemetryConfig.BlockedAttributeKeys` **env-var override** — v1 hardcodes the default list; runtime override is F16.5.

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has FIVE phases. STDOUT, FAKE-OTLP-HTTP, FILTER, and NOOP always run; REAL-COLLECTOR is gated and never claims PASS without runtime evidence. The five real-execution criteria in §5.2 each map to a unit + Challenge assertion. Span-count-zero is a hard Challenge failure.

- **CONST-039** — Challenge at `challenges/p1-f16-opentelemetry-integration/` + evidence harness at `tests/integration/cmd/p1f16_challenge/main.go`.
- **CONST-042 (No-Secret-Leak)** — `attribute_filter.go` enforces the blocklist for any user-provided attributes. `DefaultBlockedAttributeKeys` covers credential-shaped keys + prompt-body keys. Prompt body is NEVER added as a span attribute. `TestTracedLLMProvider_DoesNotEmitPromptBodyAttribute` asserts this. The Challenge's FILTER phase deliberately injects `api_key=sk-abc-123` and `prompt=<body>` and asserts they are dropped from captured exports.
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force; explicit user authorization is requested at T12 before pushing.
- **No-Mocks-In-Production (Universal Rule 2)** — `TelemetryProvider`, all three exporters, the three instrumentation sites, and the `/telemetry` slash are real. `tracetest.SpanRecorder` and the OTel `metric/sdk/metricdata` reader are TEST-ONLY scoped under `_test.go` files; the in-tree fake OTLP/HTTP receiver is the Challenge harness ONLY (under `tests/integration/cmd/p1f16_challenge/`) and is not linked from production paths.

10. Open questions resolved

Q	Answer	Resolution
Q1: scope	(B) tracing + metrics, no logs-bridge	OTel `Tracer` + `Meter` only; logs-bridge from zap deferred to F16.5
Q2: exporters	(C) three implementations	OTLP/gRPC (production), OTLP/HTTP (proxy-friendly), stdout (development); selected from `OTEL_EXPORTER_OTLP_PROTOCOL`; default no-op when no env var set
Q3: instrumentation surface	(B) LLM + tools + agent loop	LLM provider Generate/GenerateStream (counter for tokens, histogram for latency, span per call); ToolRegistry.Execute (span + counter + histogram per tool); agent loop iterations (span + counter + duration histogram); sandbox/subagent/LSP not instrumented in v1 (F16.5)
Q4: configuration	(B) standard OTel env vars	`OTEL_EXPORTER_OTLP_ENDPOINT`, `OTEL_EXPORTER_OTLP_PROTOCOL`, `OTEL_SERVICE_NAME` (default "helixcode"), `OTEL_RESOURCE_ATTRIBUTES`, plus standard `OTEL_EXPORTER_OTLP_INSECURE` / `OTEL_EXPORTER_OTLP_TIMEOUT`; no yaml/CLI flags in v1
Q5: user surface	(B) slash only	`/telemetry` slash with `status` / `show` / `flush`; no cobra subcommand

11. Non-obvious decisions (recorded for plan-time review)

1. **Decorator vs per-call instrumentation for the LLM hot path** — the spec picks the `TracedLLMProvider` decorator over editing each provider implementation. Reason: the four cloud providers (anthropic, bedrock, vertex, azure) plus ollama all implement the same `Provider` interface; a decorator at the construction site (main.go) is one wrapping line, vs five identical span+counter blocks across five files. The decorator uses Go struct embedding so adding a new method to the Provider interface does not require an F16 update.
2. **In-place vs decorator for the tool registry and agent loop** — the spec picks **in-place** for these two sites because the registry's `Execute` is already a chokepoint with five existing pre/post hooks (F07/F08/F13 + hooks-before/after); wrapping it with a separate decorator would force every test that exercises the registry to thread the decorator through. The agent loop has the same shape — `executeTaskWithLLM` is already the single LLM-call site; wrapping `BaseAgent` would require duplicating its public surface. Five lines in-place > a parallel decorator hierarchy.
3. **No-op vs error when no OTel env vars are set** — chose no-op. Reason: the OTel ecosystem's standard pattern is "telemetry off = no-op providers". Erroring out would force every CI pipeline that doesn't run a collector to set a sentinel env var, which is intrusive and unstable.
4. **Span-attribute naming convention** — chose OTel semantic-convention keys where they exist (`llm.model`, `llm.usage.prompt_tokens`, `llm.usage.completion_tokens`, `tool.name`) and `helixcode.*` prefix where they don't (`helixcode.tool.name` is wrong — collides with `tool.name`; the spec uses bare `tool.name` per OTel semconv). Metric names use snake_case prefix `helixcode_*` (per Prometheus convention which the OTel-to-Prom translator preserves).
5. **Sampling default** — left at OTel SDK default `parent-based(traces-and-error-codes)`. Users who want head sampling set `OTEL_TRACES_SAMPLER` / `OTEL_TRACES_SAMPLER_ARG` per the OTel spec; the SDK reads those automatically. F16 does not pin a sampler.
6. **Why no env-var override of `BlockedAttributeKeys`** — v1 hardcodes the default list to keep the CONST-042 floor non-bypassable in a production deploy. An OTEL_HELIXCODE_BLOCKED_ATTRIBUTES env var would let an operator weaken the blocklist by accident; F16.5 may add a strict "additional-keys-only" override. The seam (`TelemetryConfig.BlockedAttributeKeys` field) exists today for in-process customisation.
7. **Why ring buffer is stdout-only** — when an OTLP collector is wired, the user already has full backend visibility; mirroring exports into a process-local ring buffer would be wasted memory. Stdout is purely a developer mode and the ring buffer makes `/telemetry show` useful in that mode.
8. **Why no telemetry for subagent helpers in v1** — F15's `RunAsSubagent` constructs a minimal runtime that doesn't share parent state. Threading the parent's TracerProvider through the env-var payload would require encoding the parent's resource + endpoint in the helper's stdin handshake, which is a non-trivial protocol change. F16.5 will add it; v1 documents the gap loudly.